

Putka's Army

SIOP ML Competition Presentation

Team members:

Ammar Ansari, HumRRO

Daniel Barstow, HumRRO

John Little, HumRRO

Anoop Javalagi, HumRRO

Robert Wellman, HumRRO

Lilang Chen, HumRRO

Jiayi Liu, HumRRO

Karla Castillo-Guerra, HumRRO

Competition Challenge(s)



**SIOP ANNUAL
CONFERENCE**



NEW ORLEANS

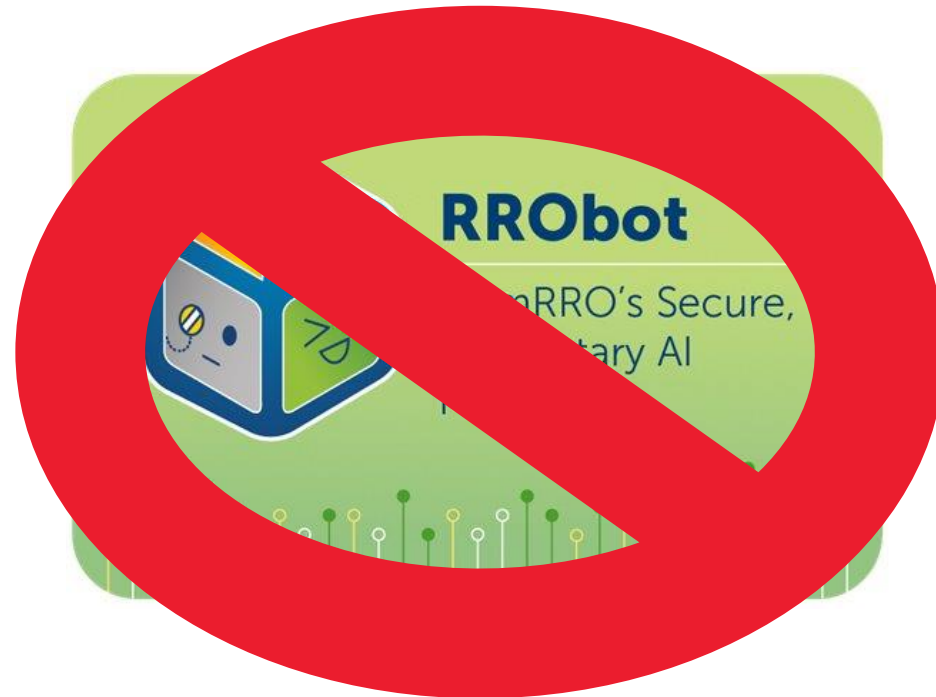


APRIL 29—MAY 2, 2026



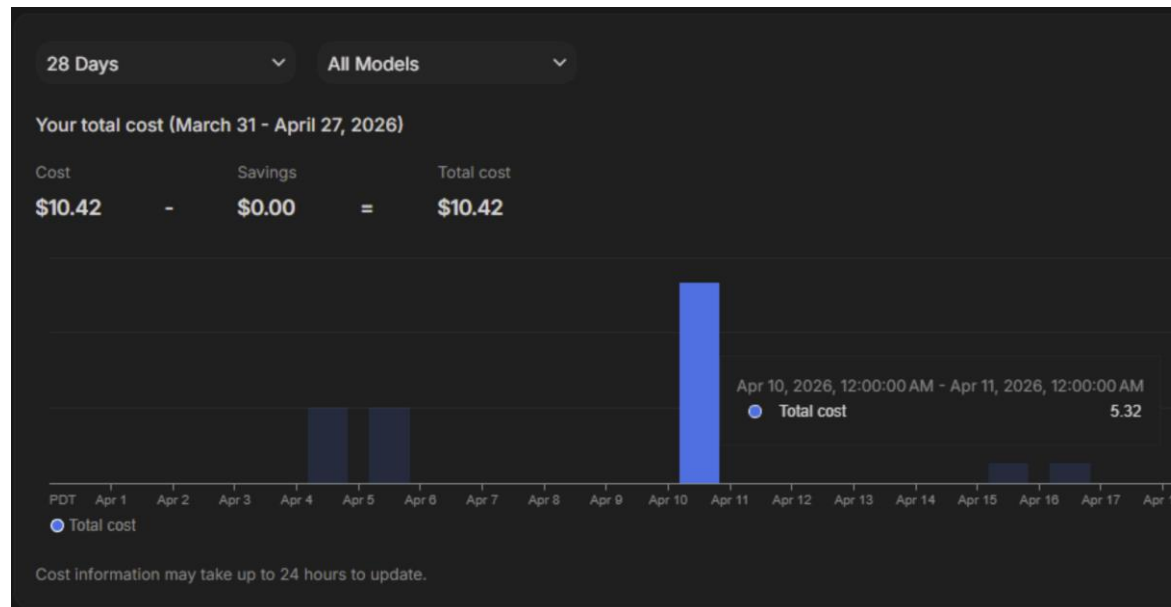
Competition Challenge(s)

- **Goal of competition:** automate the extraction of effect sizes from PDF(s), given predictor/outcome pairs and their associated research question(s).
- **Challenge #1:** Competitors were required to use LLMs that are open to the public (paid models are fine). However, this precluded our team from using RRObot, our in-house, proprietary LLM SDK with many useful features.



Competition Challenge(s) (cont.)

- **Alternative:** Decided to try Gemini, for several reasons:
 - Google updated their Gemini Deep Research Agent and Interactions API
 - Uses Gemini 3.1 Pro, offers tools such as code execution, web search, url context, and more
 - **Free tier access** to powerful models including Gemini Deep Research Agent
 - Allows for prototyping at no cost – we took great advantage of this in our trial/error process!
 - We stayed on the free tier for the entire dev phase, and only used paid models for the test set



Competition Challenge(s) (cont.)

- **Challenge #2:** Passing more than one PDF to Gemini Deep Research Agent proved challenging
 - Generally, providing Deep Research agent with more than one PDF would cause issues
 - We could have simply looped over all PDFs one at a time – however, we were limited to 5 Gemini Deep Research Agent API calls per month, per API key, on the free tier, and if we used a paid API key, Google’s docs state that each API call can be up to \$5. With ~200 PDFs across dev and test sets, this could prove exorbitantly expensive.
 - There had to be a more efficient and elegant solution....

Estimated costs ⇄

Costs vary based on the depth of research required. The agent autonomously determines how much reading and searching is necessary to answer your prompt.

- **Standard research task:** For a typical query requiring moderate analysis, the agent might use ~80 search queries, ~250k input tokens (~50-70% cached), and ~60k output tokens.
 - **Estimated total:** ~\$2.00 – \$3.00 per task
- **Complex research task:** For deep competitive landscape analysis or extensive due diligence, the agent might use up to ~160 search queries, ~900k input tokens (~50-70% cached), and ~80k output tokens.
 - **Estimated total:** ~\$3.00 – \$5.00 per task

★ **Note:** These figures are estimates based on preview rates and are subject to change.



Competition Challenge(s) (cont.)

- **Solution: Compress the data!**

- Classic compression problem: solution was to pass the PDFs to a cheaper Gemini model with a larger context window (~1 million token input context window, vs ~300k for Gemini Deep Research Agent)
 - Gemini model extracted relevant statistics from the PDF, compressing from large PDF to tiny JSON
 - Using **structured outputs** allowed for Pydantic input and output validation using JSON schema, to reduce hallucinations and improve performance of model on task
 - JSON output, with 100+ PDF's extracted statistics, could be passed to a **single Deep Research API call!**
- In this way, we reduced costs significantly (stayed on free tier) by extending the agentic workflow to include **2 separate agents**, each with a **specialized task it was best suited for.**

Competition Challenge(s) (cont.)

- **Iterative improvements:**

- Since our experimentation costs were so low (free), we were able to tweak different settings and make small improvements, one at a time:
 - The newer the model, the better, generally (e.g., Gemini 3.0 Flash > Gemini 2.5 Flash)
 - The more reasoning added, the better
 - High reasoning wasn't as reliable with the extraction agent, due to increased token usage and overflow, so settled on Medium reasoning
 - Structured outputs improved results
 - IO domain expertise was key:
 - Adding additional statistics to prompt and Pydantic JSON schema improved results (e.g., odds ratio, Chi-square, t-test, sample size for Hunter-Schmidt weighted aggregation)
 - Instructing Deep Research Agent to perform Fisher z-transformation of correlation coefficients before aggregating, and an inverse Fisher z-transform of result, improved results
 - » Unsure if method helped, or if it implicitly forced code execution (code execution rule for math operations was already in prompt, but obviously, models do not always stick to prompt exactly – hallucination is a fact of life)

High Level Overview of Pipeline



Inputs and Outputs of Pipeline

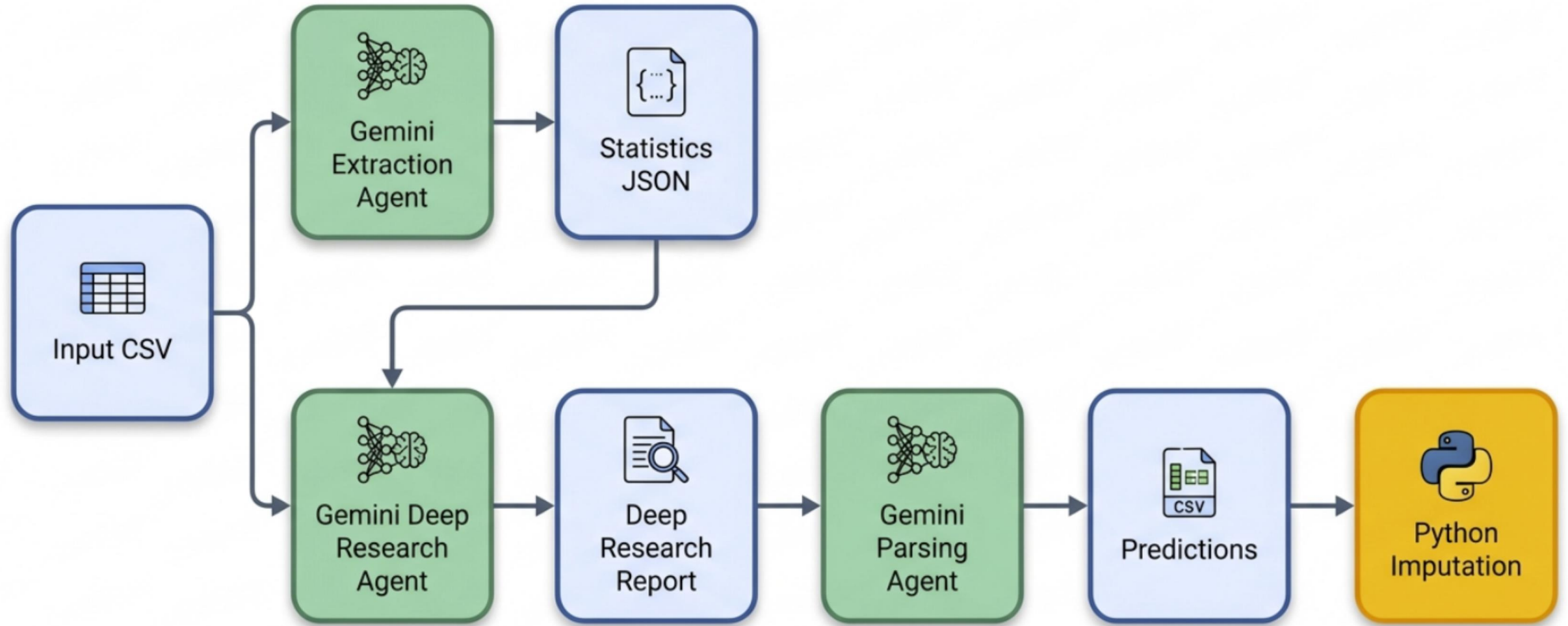
- **Inputs:**

- Study information (in simple CSV input files)
 - PDF path(s)/name(s)
 - Predictor(s)
 - Outcome(s)
 - Research question(s)
 - Construct definitions

- **Outputs:**

- JSON file(s) with extracted statistics and information (e.g., statistic type, sample size, reliability, reverse coding information, means, SD, sample size, etc)
- Google Deep Research agent markdown report
- CSV file with predictions per article

High Level Overview of Pipeline



Take Home Messages

Take Home Messages

- **Structured Outputs** seem to help reduce hallucinations
 - Used a Pydantic schema passed into API call to structure JSON output
 - Many frameworks offer this functionality (e.g., Pydantic AI, LangGraph)
- Hallucinations are fact of life - when building LLM systems, **have LLM show it's work as much as possible and at every step possible** to see where they are happening, and how to mitigate (e.g., deterministic alternative, more targeted prompt that accounts for potential issues, etc)
- **Domain expertise + engineering expertise** remains a winning combo
 - Refining prompts based on domain expert input, refining pipeline architecture based on engineer expert input

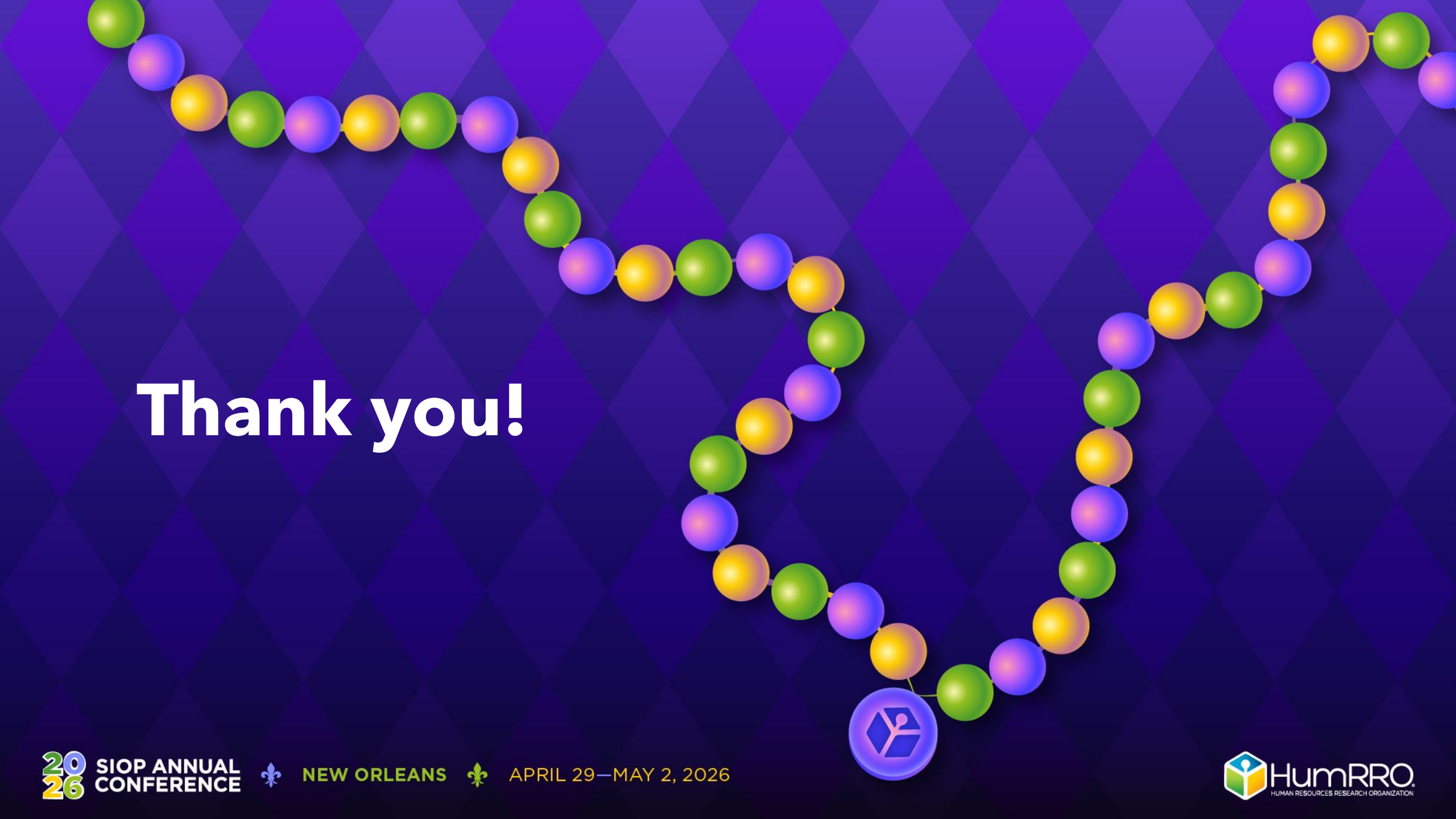


Questions?

**For more information,
please contact:**

Ammar Ansari
aansari@humrro.org





Thank you!

20
26

SIOP ANNUAL
CONFERENCE



NEW ORLEANS



APRIL 29—MAY 2, 2026



 **HumRRO**
HUMAN RESOURCES RESEARCH ORGANIZATION

Appendix

Pydantic Schema Example

```
class StatisticEntry(BaseModel):
    predictor_variable: str = Field(description="Name of the predictor/independent variable")
    criterion_variable: str = Field(description="Name of the criterion/dependent/outcome variable")
    predictor_construct: Optional[str] = Field(default=None, description="Which construct from system instructions does the predictor map to?")
    criterion_construct: Optional[str] = Field(default=None, description="Which construct from system instructions does the criterion map to?")
    statistic_type: Literal[
        "pearsons_r", "cohens_d", "hedges_g", "glass_delta", "point_biserial",
        "standardized_beta", "unstandardized_beta", "t_statistic", "f_statistic",
        "chi_square", "odds_ratio", "eta_squared", "partial_eta_squared",
        "r_squared", "mean_and_sd", "contingency_table_2x2", "other"
    ] = Field(description="Type of statistic reported")
    statistic_value: Optional[float] = Field(default=None, description="The reported value of the statistic")
    regression_type: Optional[Literal["simple", "multiple"]] = Field(default=None, description="If beta: simple (1 predictor) or multiple regression?")
    number_of_predictors: Optional[int] = Field(default=None, description="If multiple regression: how many predictors?")
    sample_size_total: Optional[int] = Field(default=None, description="Total N for this specific statistic")
    sample_size_group1: Optional[int] = Field(default=None, description="N for group 1")
    sample_size_group2: Optional[int] = Field(default=None, description="N for group 2")
    mean_group1: Optional[float] = Field(default=None, description="Mean of group 1 on criterion")
    mean_group2: Optional[float] = Field(default=None, description="Mean of group 2 on criterion")
    sd_group1: Optional[float] = Field(default=None, description="SD of group 1 on criterion")
    sd_group2: Optional[float] = Field(default=None, description="SD of group 2 on criterion")
    mean_predictor: Optional[float] = Field(default=None, description="Mean of predictor variable")
    sd_predictor: Optional[float] = Field(default=None, description="SD of predictor variable")
    mean_criterion: Optional[float] = Field(default=None, description="Mean of criterion variable")
    sd_criterion: Optional[float] = Field(default=None, description="SD of criterion variable")
    cell_a: Optional[int] = Field(default=None, description="2x2 table cell A (top-left)")
    cell_b: Optional[int] = Field(default=None, description="2x2 table cell B (top-right)")
    cell_c: Optional[int] = Field(default=None, description="2x2 table cell C (bottom-left)")
    cell_d: Optional[int] = Field(default=None, description="2x2 table cell D (bottom-right)")
    pooled_sd: Optional[float] = Field(default=None, description="Pooled SD if reported")
    se: Optional[float] = Field(default=None, description="Standard error of the statistic")
    confidence_interval_lower: Optional[float] = Field(default=None, description="Lower 95% CI")
    confidence_interval_upper: Optional[float] = Field(default=None, description="Upper 95% CI")
    p_value: Optional[float] = Field(default=None, description="P-value")
    degrees_of_freedom: Optional[float] = Field(default=None, description="df (for t-test, chi-square)")
    degrees_of_freedom_numerator: Optional[float] = Field(default=None, description="Numerator df (for F-statistic)")
    degrees_of_freedom_denominator: Optional[float] = Field(default=None, description="Denominator df (for F-statistic)")
    reliability_predictor: Optional[float] = Field(default=None, description="Cronbach's alpha of predictor")
    reliability_criterion: Optional[float] = Field(default=None, description="Cronbach's alpha of criterion")
    is_reverse_coded: Optional[bool] = Field(default=None, description="True if relationship direction needs flipping based on construct definitions")
    reverse_code_reason: Optional[str] = Field(default=None, description="Why this needs reverse coding, if applicable")
```



Competition Challenge(s) (cont.)

- **Project inspiration:** For AWS Imagine Grant work (ask Dan Putka about this!), Brian Prost (RRObot architect) went into a lot of detail about LLM agents, and how they differ from regular chat instances (namely, autonomous LLMs operating asynchronously, usually with tools [e.g., code execution or web search] or custom defined functions to achieve tasks in loops, essentially)



Competition Challenge(s) (cont.)

- **Personal goal:** Use this year's SIOP ML competition as a vehicle to learn more about these sort of agentic systems and how to implement them. I've done competitions like this in the past (including this one), and they are an incredible learning opportunity. Gamification + competition = accelerated learning (at least, if you enjoy competition).
- **Enter Google Gemini Deep Research Agent**

<https://arxiv.org/pdf/2601.12346>

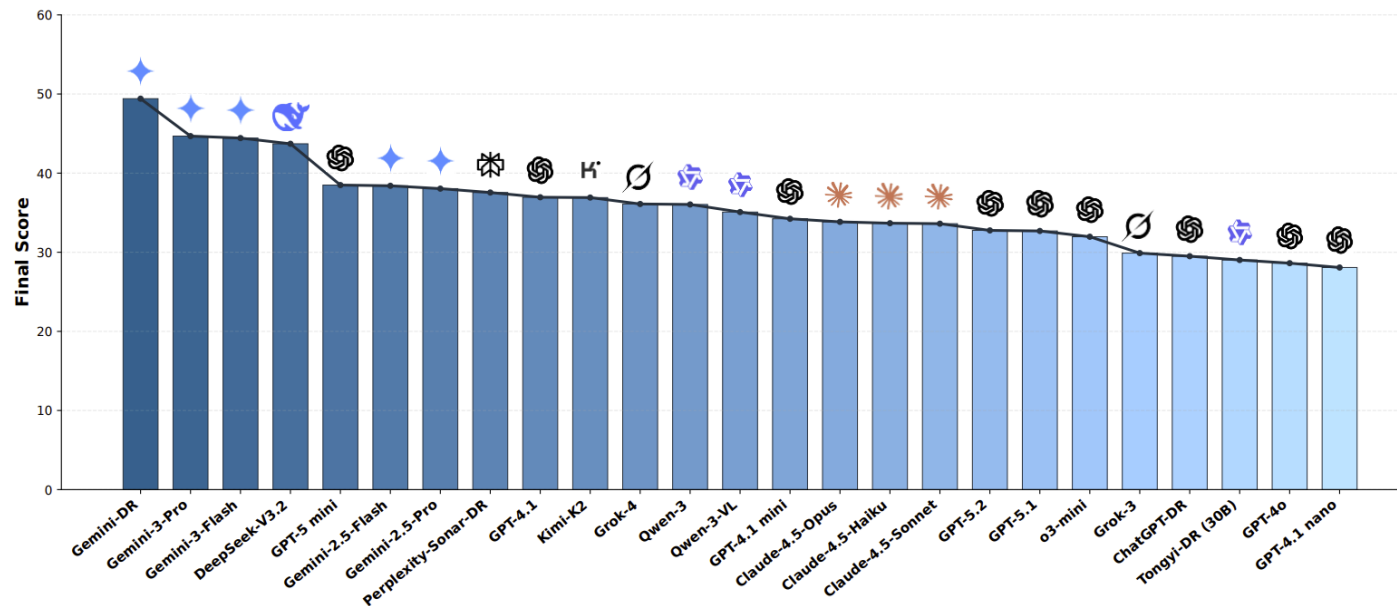
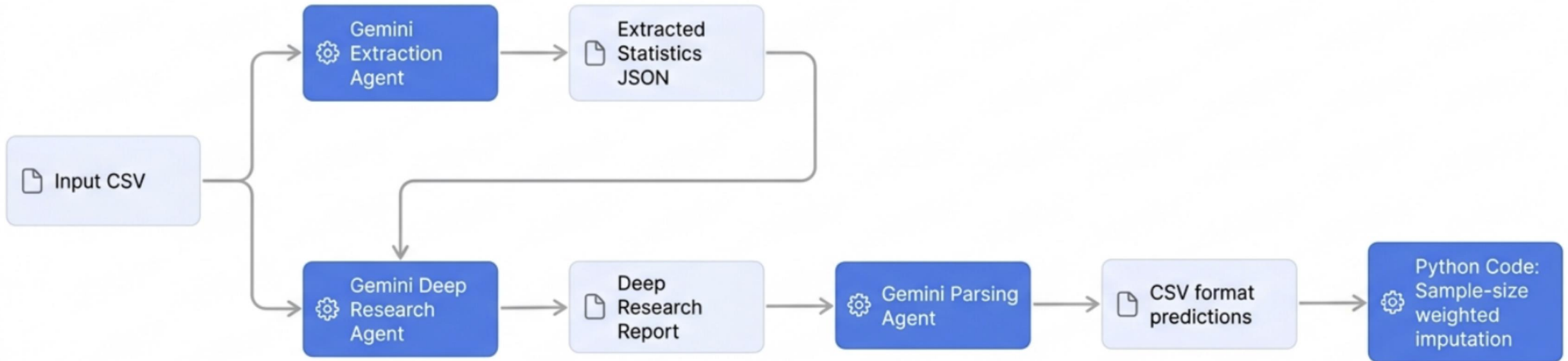


Figure 1 Overall MMDR-Bench score (0–100; higher is better) on 140 tasks for representative tool-using LLMs and Deep Research systems, ranked by score.

High Level Overview of Pipeline



High Level Overview of Pipeline

- Google Deep Research Agent
- Gemini parses PDF for any statistics describing research question for predictor/outcome and saves as JSON file.
- Deep Research agent build prompt and all json files are included in DR prompt as text. DR agent has code execution and web search tools, and performs the combination of stats.
- Final call to gemini parses DR agent markdown output and produces table
- Last bit of python code validates and handles missing values via sample-size weighted mean of studies with same constructs/research question (only 1 missing value out of 66 articles on test set).